Multi-Objective Evolutionary Algorithms based on Decomposition (MOEA/D)

Md. Rashidul Islam 2205072
Fathum Mobin 2205073
Md. Riyadun Nabi 2205076

23 February 2026

TODAY WE WILL TALK ABOUT

MOEA/D

Multi-**O**bjective **E**volutionary **A**lgorithm based on **D**ecomposition

TODAY WE WILL TALK ABOUT

MOEA/D

Multi-**O**bjective **E**volutionary **A**lgorithm based on **D**ecomposition

A powerful **evolutionary algorithm** for solving **multi-objective optimization** problems.

But before we understand the algorithm...

But before we understand the algorithm...

we must understand
what multi-objective optimization is.

What Is the Best Machine Learning Model?

Three models trained on the same dataset. Same task. Different trade-offs.

Model A

Accuracy: **98%**
Speed: Slow

Model B

Accuracy: **90%**
Speed: Very Fast

Model C

Accuracy: **95%**
Speed: Medium

What Is the Best Machine Learning Model?

Three models trained on the same dataset. Same task. Different trade-offs.

Model A

Accuracy: **98%**
Speed: Slow

Model B

Accuracy: **90%**
Speed: Very Fast

Model C

Accuracy: **95%**
Speed: Medium

Which model is optimal?

Why Does This Matter? Real-World Trade-Offs

Most decisions in the real world force you to balance competing priorities.

Cost vs Quality

Speed vs Safety

Risk vs Return

Accuracy vs Interpretability

Why Does This Matter? Real-World Trade-Offs

Most decisions in the real world force you to balance competing priorities.

Cost vs Quality

Speed vs Safety

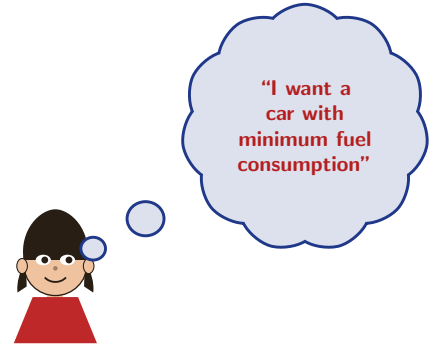
Risk vs Return

Accuracy vs Interpretability

Most real-world problems involve conflicting objectives.

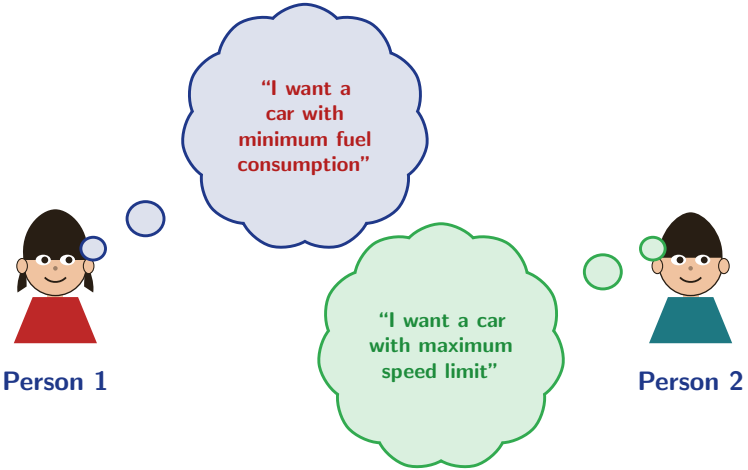
Improving one goal often means compromising another.

Real-World Example: Buying a Car

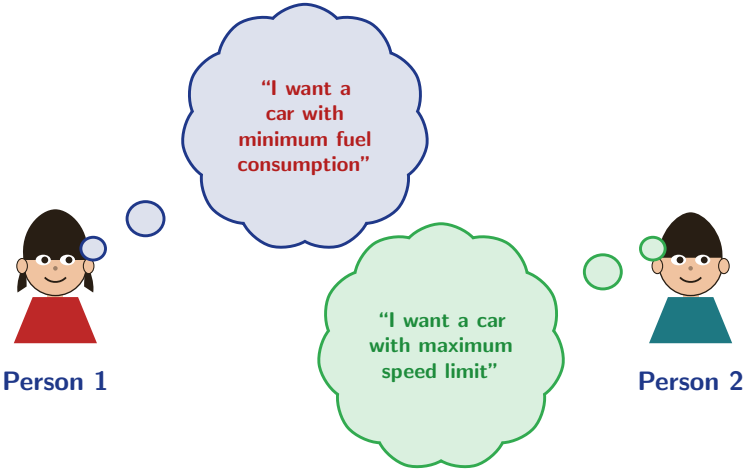


Person 1

Real-World Example: Buying a Car



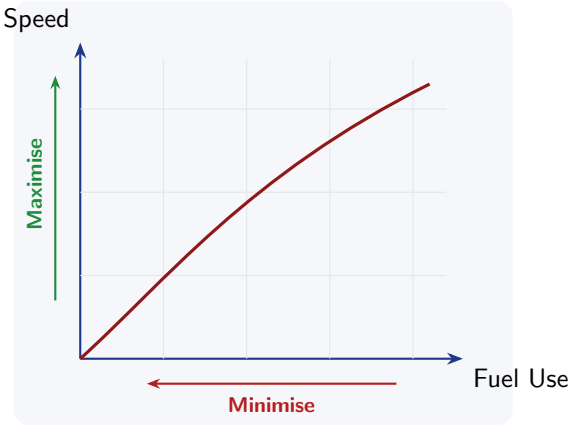
Real-World Example: Buying a Car



Both objectives matter — but they conflict with each other!

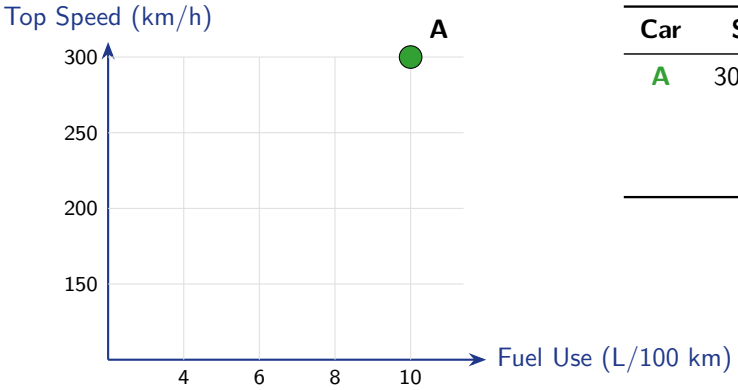
A Real-World Example: Buying a Car

Conflict:
More speed
$\Rightarrow$ more fuel



Four Cars on the Shortlist

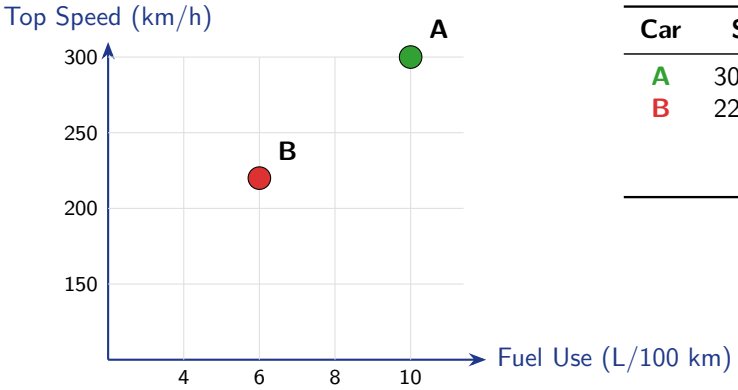
They agree on four candidate cars. Can we shrink the list?



Car	Speed	Fuel
A	300 km/h	10 L/100

Four Cars on the Shortlist

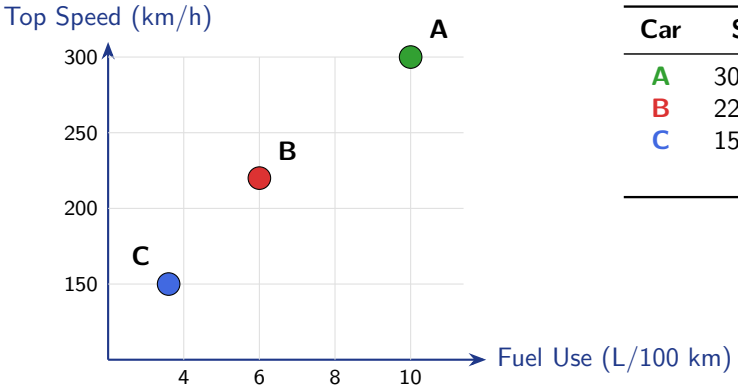
They agree on four candidate cars. Can we shrink the list?



Car	Speed	Fuel
A	300 km/h	10 L/100
B	220 km/h	6 L/100

Four Cars on the Shortlist

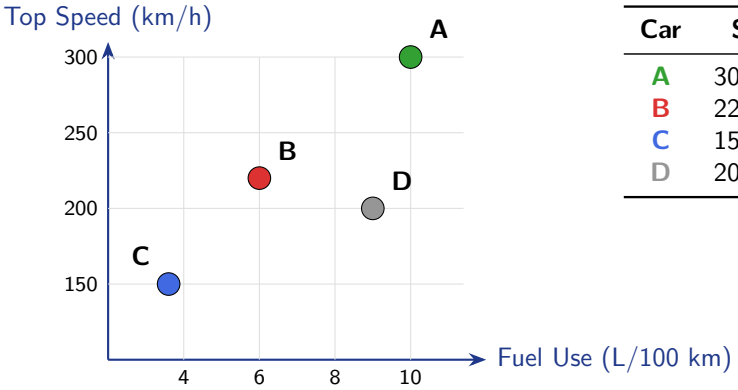
They agree on four candidate cars. Can we shrink the list?



Car	Speed	Fuel
A	300 km/h	10 L/100
B	220 km/h	6 L/100
C	150 km/h	4 L/100

Four Cars on the Shortlist

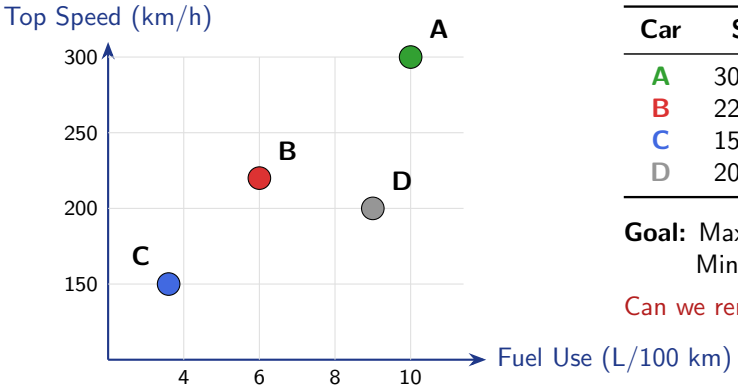
They agree on four candidate cars. Can we shrink the list?



Car	Speed	Fuel
A	300 km/h	10 L/100
B	220 km/h	6 L/100
C	150 km/h	4 L/100
D	200 km/h	9 L/100

Four Cars on the Shortlist

They agree on four candidate cars. Can we shrink the list?



Car	Speed	Fuel
A	300 km/h	10 L/100
B	220 km/h	6 L/100
C	150 km/h	4 L/100
D	200 km/h	9 L/100

Goal: Maximise speed,
Minimise fuel use.

Can we remove any car?

Dominance & Non-Domination

The Key Concept: Dominance

A solution x_1 **dominates** another solution x_2 (written $x_1 \prec x_2$)
if and only if *both* of the following conditions are met:

The Key Concept: Dominance

A solution x_1 **dominates** another solution x_2 (written $x_1 \prec x_2$) if and only if *both* of the following conditions are met:

Condition 1 — No Worse in All Objectives:

x_1 is **no worse** than x_2 in *every* objective.

The Key Concept: Dominance

A solution x_1 **dominates** another solution x_2 (written $x_1 \prec x_2$) if and only if *both* of the following conditions are met:

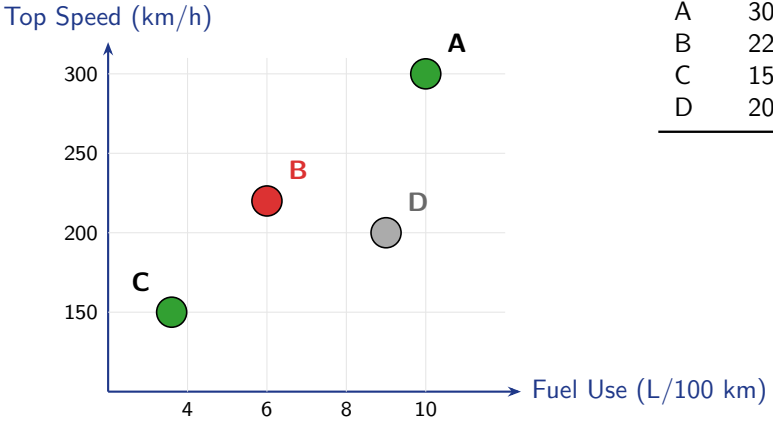
Condition 1 — No Worse in All Objectives:

x_1 is **no worse** than x_2 in *every* objective.

Condition 2 — Strictly Better in at Least One:

There exists **at least one** objective where x_1 is **strictly better** than x_2.

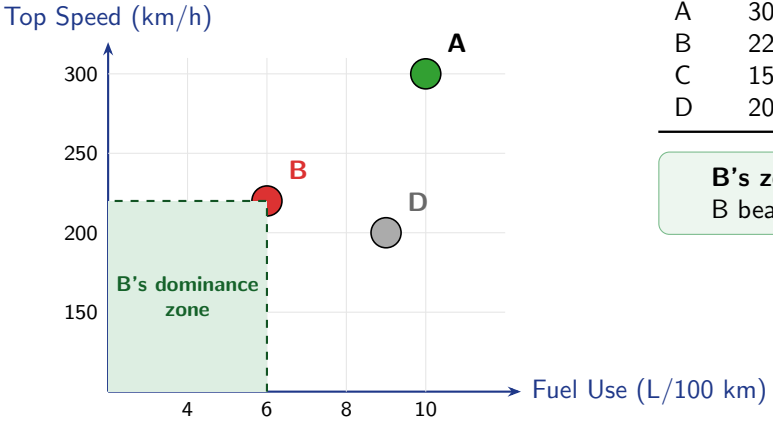
Visualising Dominance: All Four Cars



Our four candidates:

Car	Speed	Fuel
A	300 km/h	10 L
B	220 km/h	6 L
C	150 km/h	4 L
D	200 km/h	9 L

Visualising Dominance: All Four Cars

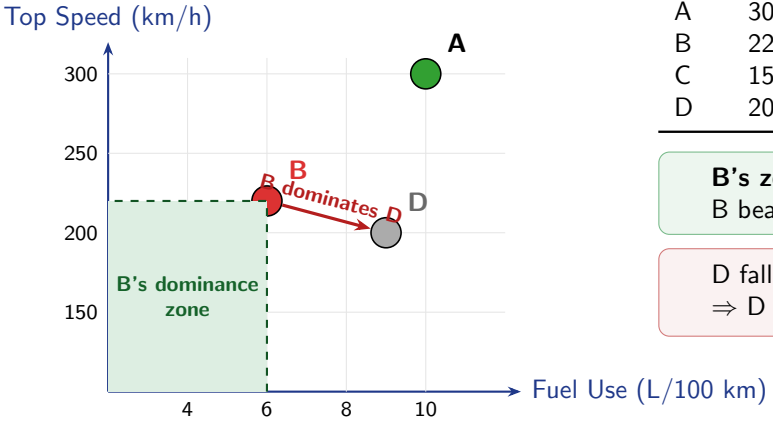


Our four candidates:

Car	Speed	Fuel
A	300 km/h	10 L
B	220 km/h	6 L
C	150 km/h	4 L
D	200 km/h	9 L

B's zone = all solutions B beats on *both* axes.

Visualising Dominance: All Four Cars



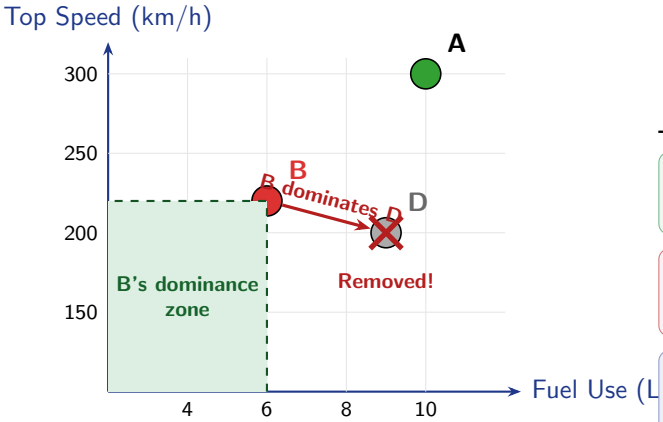
Our four candidates:

Car	Speed	Fuel
A	300 km/h	10 L
B	220 km/h	6 L
C	150 km/h	4 L
D	200 km/h	9 L

B's zone = all solutions B beats on *both* axes.

D falls *inside* B's zone
⇒ D is dominated by B.

Visualising Dominance: All Four Cars



Our four candidates:

Car	Speed	Fuel
A	300 km/h	10 L
B	220 km/h	6 L
C	150 km/h	4 L
D	200 km/h	9 L

B's zone = all solutions B beats on *both* axes.

D falls *inside* B's zone
⇒ D is dominated by B.

A, B, C are all outside each other's zones.
None dominates the others!

The Remaining Cars Form the Pareto Front

Can A, B, or C dominate each other?

Pair	Speed	Fuel	Result
A vs B	A ✓	B ✓	**No dominance**
A vs C	A ✓	C ✓	**No dominance**
B vs C	B ✓	C ✓	**No dominance**

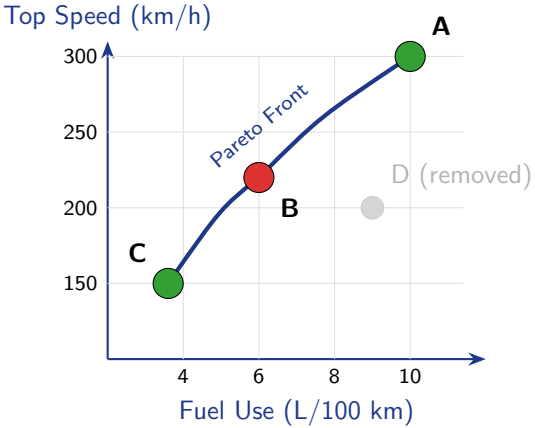
The Remaining Cars Form the Pareto Front

Can A, B, or C dominate each other?

Pair	Speed	Fuel	Result
A vs B	A ✓	B ✓	**No dominance**
A vs C	A ✓	C ✓	**No dominance**
B vs C	B ✓	C ✓	**No dominance**

Each car excels at something the others don't.

All three are non-dominated!



The Goal of Multi-Objective Optimization

The Fundamental Goal

Multi-objective optimization aims to **find (or approximate) the entire non-dominated set** — giving the decision-maker the full picture of trade-offs.

Multiple Conflicting Objectives → defines → **Dominance Concept** → identifies → **Non-Dominated Set (Pareto Front)**

What if there are millions of solutions?

The Challenge

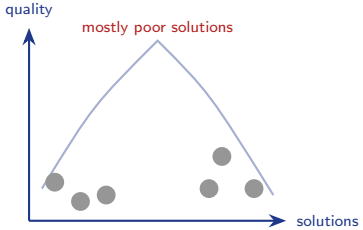
Imagine you have **millions of possible solutions** to a problem. You cannot try them all. How do you find a good one *fast*?

What if there are millions of solutions?

The Challenge

Imagine you have **millions of possible solutions** to a problem. You cannot try them all. How do you find a good one *fast*?

A bad strategy: Random Search



Picking random solutions wastes most attempts on bad regions.

What if there are millions of solutions?

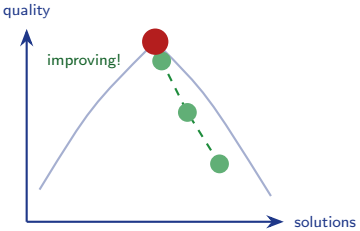
The Challenge

Imagine you have **millions of possible solutions** to a problem. You cannot try them all. How do you find a good one *fast*?

A bad strategy: Random Search

Picking random solutions wastes most attempts on bad regions.

A smarter strategy: Learn & Improve



Use what you've learned so far to guide the next attempt.

What if there are millions of solutions?

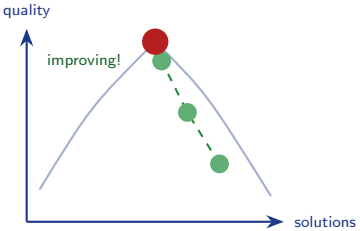
The Challenge

Imagine you have **millions of possible solutions** to a problem. You cannot try them all. How do you find a good one *fast*?

A bad strategy: Random Search

Picking random solutions wastes most attempts on bad regions.

A smarter strategy: Learn & Improve



Use what you've learned so far to guide the next attempt.

Evolutionary Algorithms do exactly this — guided, improving search.

Three Steps — The Evolutionary Cycle

Step 1 — Selection

Three Steps — The Evolutionary Cycle

Step 1 — Selection

↓

Step 2 — Variation
(Crossover + Mutation)

Three Steps — The Evolutionary Cycle

Step 1 — Selection

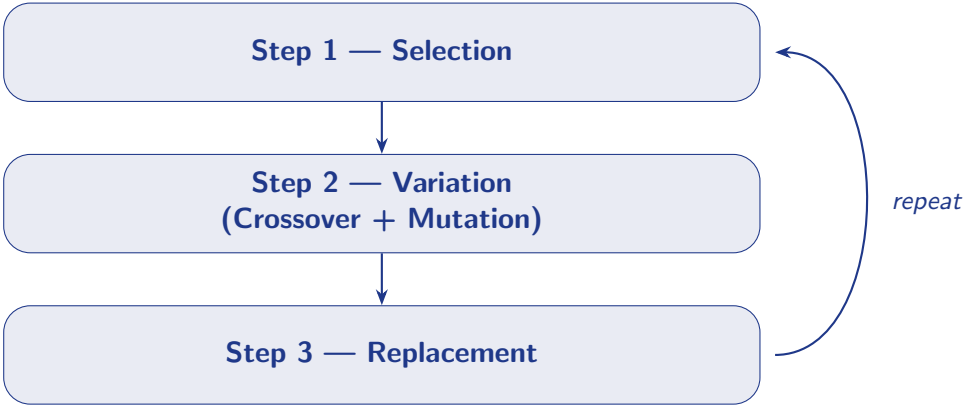
↓

Step 2 — Variation
(Crossover + Mutation)

↓

Step 3 — Replacement

Three Steps — The Evolutionary Cycle



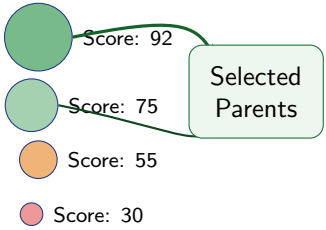
Step 1 in Detail: Selection

What is Selection?

We look at every solution in the population and give **better-scoring solutions a higher chance** of being chosen as parents. Worse solutions can still be chosen — just less often.

Why not always pick the single best?
Because the best-so-far might not lead to the globally best solution. Keeping some variety avoids getting stuck in a local "good-enough" trap.

Our Population



Bigger = better score

Step 2a: Crossover — Mixing Parents

What is Crossover?

Take **two parent solutions** and combine parts of each to create a **child solution**. The child gets some traits from Parent A and some from Parent B.

Parent A

Engine: 2.0L
Weight: 1100kg
Tyres: 205mm

Parent B

Engine: 3.5L
Weight: 1500kg
Tyres: 225mm

some traits → *some traits* →

Engine: 2.0L
Weight: 1500kg
Tyres: 205mm

Child (crossover)

Step 2b: Mutation — Adding a Little Randomness

What is Mutation?

After crossover, we **randomly change** one or a few values in the child solution by a small amount.

Why do we need this?
Crossover can only combine traits that *already exist* in the population. Mutation **introduces brand new values** — so the algorithm can explore regions it has never visited.

Child after crossover

Engine: 2.0L
Weight: 1500kg
Tyres: **205mm**

↓ **Mutation! tweak one value**

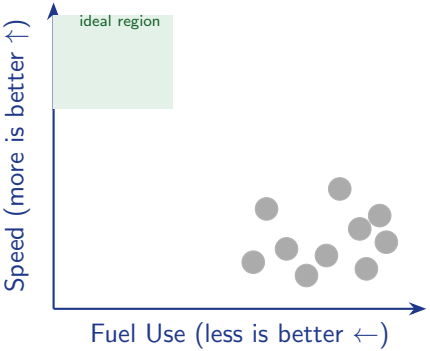
Engine: 2.0L
Weight: 1500kg
Tyres: **215mm**

After mutation

Let's Watch It Happen: A Visual Simulation

Generation 0 — random start

All solutions are poor — scattered with high fuel use, low speed



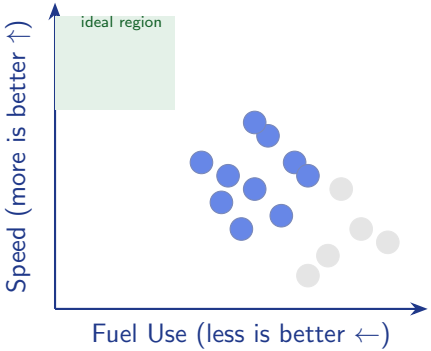
Generation 0

We **start randomly**. Most solutions are bad — high fuel use and low speed. But we have a wide spread to work with.

The gray dots are our initial population, scattered with no structure.

Let's Watch It Happen: A Visual Simulation

Generation 1 — first improvement



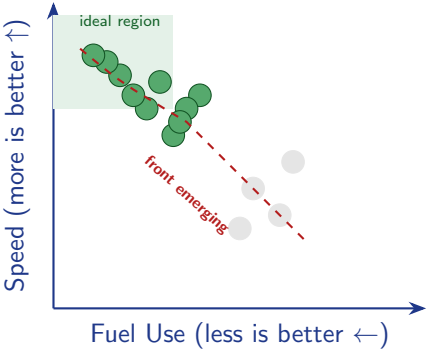
Selection + crossover pulled the cloud toward better regions

Generation 1

Selection chose the slightly-better parents. **Crossover** mixed them. The whole cloud has *shifted* toward lower fuel use and higher speed.

Let's Watch It Happen: A Visual Simulation

Generation 2 — converging fast

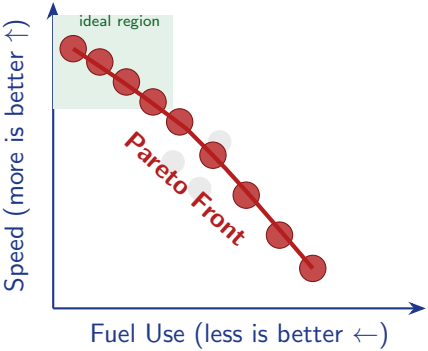


Generation 2

The population has **gathered near the frontier**. A Pareto front is beginning to take shape — diverse trade-off solutions are appearing.

Let's Watch It Happen: A Visual Simulation

Converged — Pareto Front Discovered!



Done!

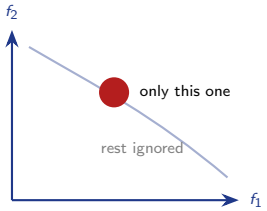
After many generations we have a **full spread of non-dominated solutions** — the Pareto Front. Each red dot is a valid, excellent trade-off.

No single "winner" — the decision-maker chooses which trade-off fits their needs.

Why Is This Perfect for Multi-Objective Problems?

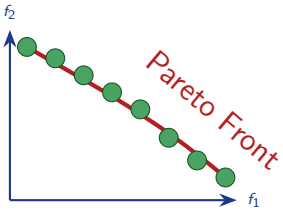
Recall: In a multi-objective problem there is no single "best" — there is a *set* of best trade-offs (the Pareto Front).

Classical optimisers find one solution



They require you to *decide* on a weighting up front. If priorities change, you must re-run from scratch.

EAs naturally find many solutions



The **population** maps naturally to a *set* of trade-off solutions. One run gives the full picture.

MOEA/D: The Idea of Decomposition

We saw: Evolutionary Algorithms approximate the Pareto front

In MOEA/D, we do not search the entire front globally

We decompose one multi-objective problem into many simple subproblems

The Idea of Decomposition

Here comes the idea of Decomposition

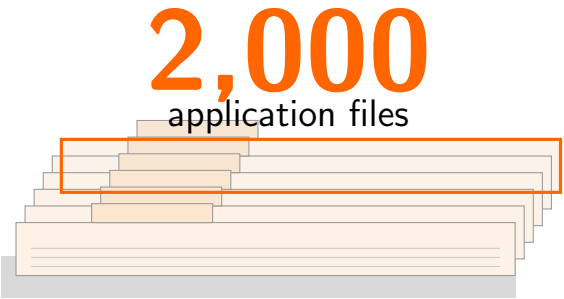
Let us go through a story from a university

It's That Time of Year...

University Scholarship Selection Season

You are on the **Selection Committee**
Your task: choose the most deserving candidates.

The Stack on Your Desk



Two thousand students
each hoping for a scholarship.

You Open the First Few Files...

Alice

CGPA: **3.98**
Research: avg
Leadership: avg
Financial need: low

Bob

CGPA: 3.40
Research: **4 papers**
Leadership: avg
Financial need: low

Carol

CGPA: 3.50
Research: avg
Leadership: **excellent**
Financial need: avg

...And More Files

Dave

CGPA: 3.10
Research: low
Leadership: avg
Financial need:
very high

Eva

CGPA: 3.70
Research: **3 papers**
Leadership: **good**
Financial need: avg

$\cdots$ *and 1,995 more just like these.*

The Obvious Question

“JUST RANK THEM!!!”

Assign a score to each applicant.
Sort from best to worst. Pick the top ones.

Sounds simple, right?

But Wait — What's the Score?

To give a single score, you need
to combine all four criteria.

$$\text{score} = w_1 \cdot \text{CGPA} + w_2 \cdot \text{Research} + w_3 \cdot \text{Leadership} + w_4 \cdot \text{Need}$$

What should w_1, w_2, w_3, w_4 be?

Nobody Agrees on the Weights

Prof. A:
"CGPA is everything!"

Prof. B:
"Research matters most."

Prof. C:
"We need leaders!"

Prof. D:
"Financial need is priority."

Every committee member has a **different** *notion of "best".*

Okay, Let's Try. Who's Better: Alice or Bob?

Alice

CGPA: **3.98**
Research: 0 papers
Leadership: Avg
Need: Low

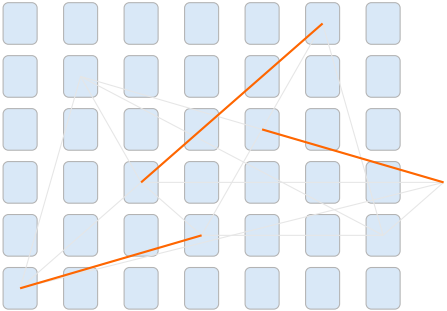
vs

Bob

CGPA: 3.40
Research: **4 papers**
Leadership: Avg
Need: Low

Alice wins on CGPA.
Bob wins on Research.
Nobody dominates the other.

Now Do That for 2,000 Applicants



$\approx$ 2,000,000
comparisons!

Every applicant must be compared with every other applicants.
Computationally expensive and conceptually messy.

And Even If You Do It...

Final Top 10 Selected:

#1 Alice CGPA: 3.98	#2 James CGPA: 3.97	#3 Priya CGPA: 3.96	#4 Lena CGPA: 3.95	#5 Omar CGPA: 3.94
#6 Sara CGPA: 3.93	#7 Chen CGPA: 3.92	#8 Fatima CGPA: 3.91	#9 Ravi CGPA: 3.90	#10 Nina CGPA: 3.89

Problem: Every selected candidate has a high CGPA — and nothing else. Bob (4 papers), Carol (leadership), Dave (financial need) — all rejected.

The Real Issue

There is no single "Best" applicant.
There are only applicants who
are best in a certain way.

Alice is best if you care about **CGPA**.
Bob is best if you care about **Research**.
Dave is best if you care about **Financial need**.

Forcing a single global ranking destroys this structure.

A Better Idea Starts to Form...

What if we **stopped** trying to build one global ranking...

One hard global ranking
(expensive, unfair, not diverse)

↓

Several focused sub-selections
(one per preference type)

Each sub-problem is simple. Together, they cover everything.

Two Questions That Change Everything

What if we stop building one global ranking?

What if we evaluate under different preferences instead?

These two questions lead us directly to the core idea behind **MOEA/D**.

And Together, They Cover Everything

Bob — Research
4 publications

Carol — Leadership
Club president, 3 NGOs

Dave — Need-Based
Family income: very low

Eva — Balanced
Good across all criteria

Diverse. Fair. No single winner dominates.

The Key Insight

One global ranking $\times$

Multiple preference-based rankings $\checkmark$

Each sub-ranking is **simple** to compute.
Together, they are **complete** —
covering all preference directions.

This Is Called: Decomposition

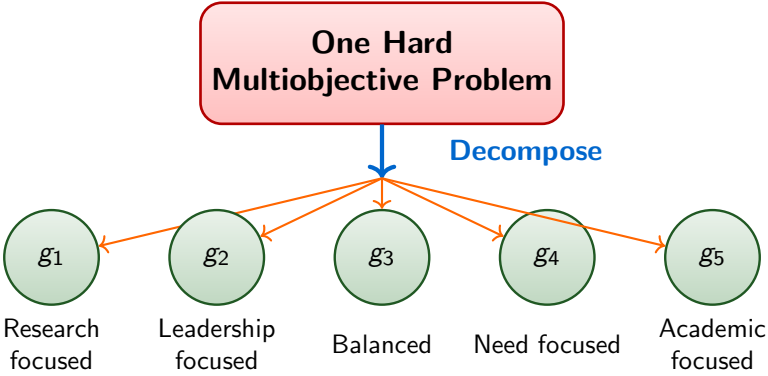
Instead of one hard problem,
solve many simple ones.

$\longrightarrow$ **Decomposition** $\longleftarrow$

Breaking a multiobjective optimization problem
into multiple scalar subproblems, each with a different preference.

One multiobjective problem $\Rightarrow$ many scalar subproblems

Visualizing Decomposition



Each g_i is a **scalar** optimization problem — easy to solve individually.

A Second Story: Buying a Phone

You want to buy a new smartphone.

You care about two things:

1. **Price** — as low as possible
2. **Camera Quality** — as high as possible.

You Walk Into the Store...

Phone A
$200
Camera:
2/5
Budget pick

Phone B
$450
Camera:
4/5
Mid-range

Phone C
$1,100
Camera:
5/5
Flagship

Phone D
$500
Camera:
2/5
Overpriced?

Which One Is "Best"?

It depends entirely on your **preference**.

On a tight budget?
Phone A wins

Want the best camera?
Phone C wins

Want a balance?
Phone B wins

And what about phone D? Phone D is dominated — nobody would choose it.

Phone D Is Dominated

Phone A
$200
cam 2/5

Phone B
$450
cam 4/5

Better camera
Lower price

Phone D
$500
cam 2/5

Phone D is dominated. Phone B is better in both criteria.
Dominated options can be safely ignored.

The Phones That Remain: Non-Dominated

Phone A
$200
cam 2/5
Best price

Neither dominates

Phone B
$450
cam 4/5
Balanced

Neither dominates

Phone C
$1,100
cam 5/5
Best camera

These three phones are **non-dominated** — the best trade-offs available.
Together they form the **PARETO FRONT**.

Back to the Phone Store: Each Preference = One Subproblem

Phone A
Price: **$200**
Camera: 2/5
Best price

↓

Budget focus
g_1: **minimize price**

Phone B
Price: $450
Camera: **4/5**
Balanced choice

↓

Balanced preference
g_3: **equal weight**

Phone C
Price: $1,100
Camera: **5/5**
Best camera

↓

Camera focus
g_5: **maximize quality**

Different preferences lead to different phones — and different scalar subproblems

Introducing the Weight Vector λ

A weight vector $\lambda = (\lambda_1, \lambda_2)$
encodes a preference over objectives

$\lambda_1 + \lambda_2 = 1$ (like percentages)

$\lambda = (0.1, 0.9)$
Price-focused
"90% Camera,
10% Price"
finds Phone C

$\lambda = (0.5, 0.5)$
Perfectly Balanced
"50% Camera,
50% Price"
finds Phone B

$\lambda = (0.9, 0.1)$
Camera-focused
"10% Camera,
90% Price"
finds Phone A

Each λ defines one scalar subproblem
and one focused search direction

Two Classic Ways to Scalarize

Weighted Sum

Combine objectives as a weighted average

$$\text{Score} = \lambda_1 f_1 + \lambda_2 f_2$$

Simple • Fast • Convex fronts

or

Tchebycheff

Minimize the worst gap to the ideal point

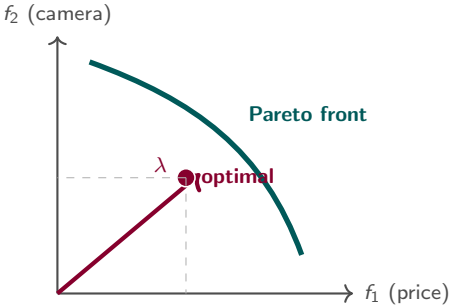
Focus on the weakest objective

Works for non-convex fronts

More robust

In both cases, λ converts many objectives into one scalar score

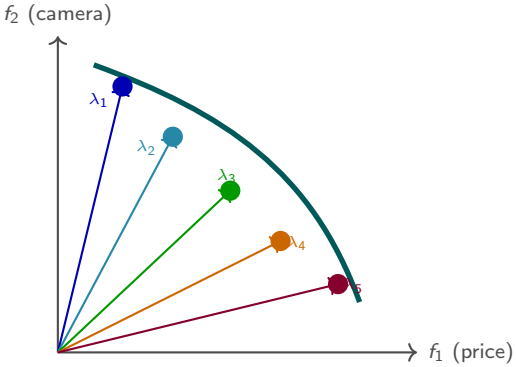
λ as a Direction: Geometric Intuition



Each λ defines a **search direction**.

The best solution
along that direction
lies on the **Pareto front**.

Many λ Directions $\rightarrow$ Many Pareto Points

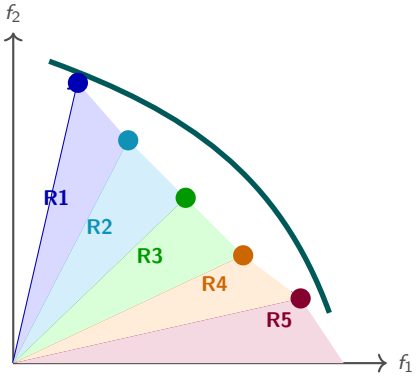


Each λ_i defines one subproblem

$\lambda_1 = (0.1, 0.9)$ camera-focused
$\lambda_3 = (0.5, 0.5)$ balanced
$\lambda_5 = (0.9, 0.1)$ price-focused

Many directions $\rightarrow$ many solutions covering the Pareto front

Each λ "Owns" One Region of the Pareto Front

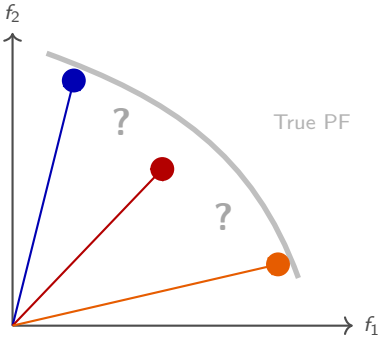


Each λ_i owns one region

Each subproblem searches only inside its region and finds its best Pareto solution

All regions together $\Rightarrow$ full Pareto front coverage

Few Weight Vectors → Sparse Coverage



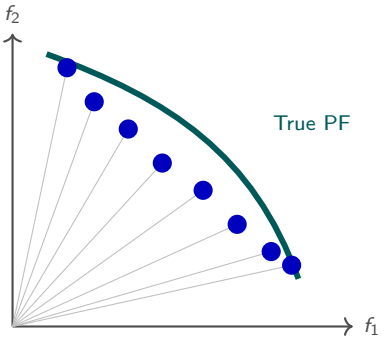
Too Few Weight Vectors

⇒ only **3 Pareto solutions**

⇒ **large gaps** in the front

Many trade-offs are missed

Many Weight Vectors → Uniform Coverage



Many Weight Vectors

$\Rightarrow$ many Pareto solutions

$\Rightarrow$ uniform coverage

All trade-off regions represented

Dense λ $\Rightarrow$ high diversity

Why Is Decomposition So Powerful? #1: Local Focus

Without Decomposition

Every solution competes against *all* others.

Hard to make progress — too many directions pulling at once.

Like trying to be the best at everything simultaneously.

vs.

With Decomposition

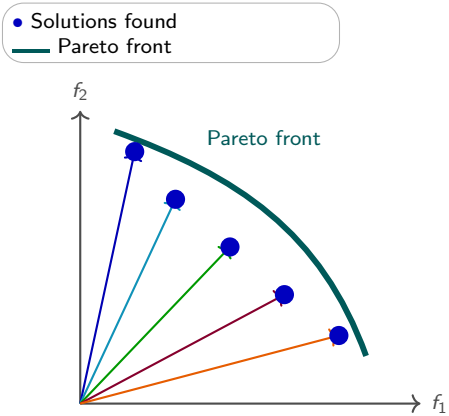
Each subproblem has *one clear direction*.

Search concentrates on improving along that direction.

Like being the best *budget phone*. Focused.

Local focus $\Rightarrow$ each subproblem becomes easier to optimize

Why Is Decomposition So Powerful? #2: Natural Diversity



Natural Diversity

Different subproblems search in different directions.

Solutions spread across the entire Pareto front.

No extra mechanism needed

Diversity is built into decomposition

Why Is Decomposition So Powerful? #3: No Global Ranking Needed

Global Ranking

Requires comparing every solution to every other.

Who is "better" when one excels at f_1 and another at f_2?

Expensive. Ambiguous.

vs.

Decomposition

Each subproblem has a scalar objective g.

Just minimize g. No cross-comparisons between subproblems.

Simple. Scalable.

Each subproblem becomes a simple optimization task — no dominance sorting required

All of this — directions, subproblems, scalarization, coverage — leads to one conclusion:

Decomposition

is the foundation of
MOEA/D

Every design decision in MOEA/D follows from this one idea

So Far: Subproblems. Next: Making Them Cooperate.

Subproblems

$g_1, g_2, \ldots, g_N$

Each has a direction λ_i

+

Cooperation

Neighboring subproblems share information

↓

MOEA/D

Up next: How do subproblems cooperate?

MOEA/D Philosophy

Two Key Ideas:

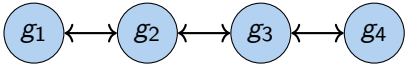
Two Key Ideas:

1. **Decomposition:** Break into simple problems

Two Key Ideas:

1. **Decomposition:** Break into simple problems

2. **Collaboration:** Let similar problems help each other



Share information!

The 5 Friends - Each with Different Car Preferences

Understanding MOEA/D: The 5 Friends Story

The 5 Friends - Each with Different Car Preferences

Friend 1 Quality first (90% Quality-10% Price)

Friend 2 Good quality (70% Quality-30% Price)

Friend 3 Balanced (50% Quality-50% Price)

Friend 4 Cheaper car (30% Quality-70% Price)

Friend 5 Cheapest! (10% Quality-90% Price)

Understanding MOEA/D: The 5 Friends Story

The 5 Friends - Each with Different Car Preferences

Friend 1 Quality first (90% Quality-10% Price)

Friend 2 Good quality (70% Quality-30% Price)

Friend 3 Balanced (50% Quality-50% Price)

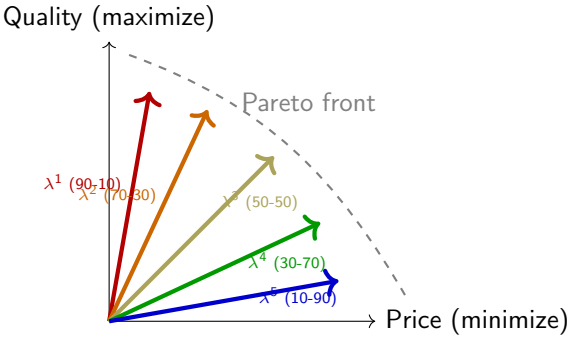
Friend 4 Cheaper car (30% Quality-70% Price)

Friend 5 Cheapest! (10% Quality-90% Price)

Each friend = one subproblem

$\Rightarrow$ with a different weight vector λ

INITIALIZATION:

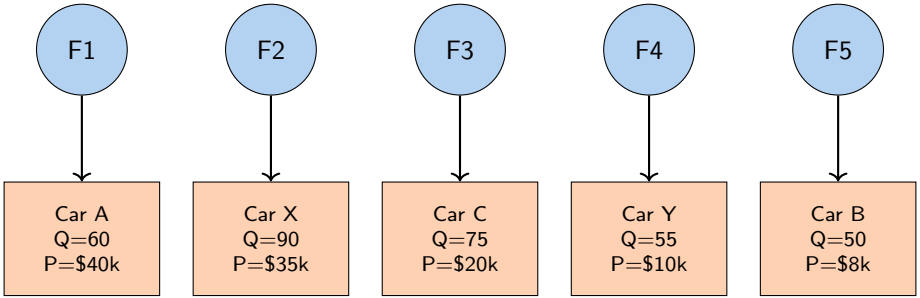


Step 1: Random Initial Solution

Each friend randomly picks one car from the **design space**

Step 1: Random Initial Solution

Each friend randomly picks one car from the **design space**



This creates the **initial population** of solutions

STEP 2: Crossover+Mutation — Combining Ideas From Neighbors

Friend 3 takes parts from both neighbors' cars to build a new one

Parent 1 (F2's neighbor)

F2's Car X

Engine: V8
Seats: Leather
Brand: Premium
Price: $35k

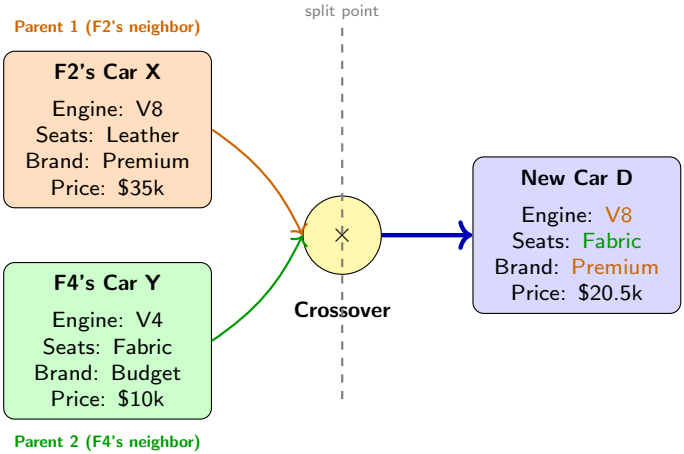
F4's Car Y

Engine: V4
Seats: Fabric
Brand: Budget
Price: $10k

Parent 2 (F4's neighbor)

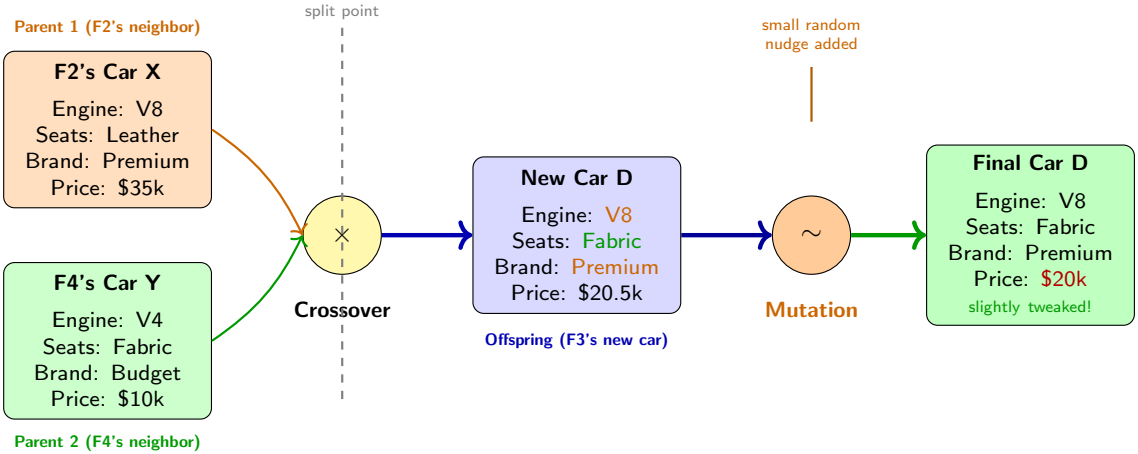
STEP 2: Crossover+Mutation — Combining Ideas From Neighbors

Friend 3 takes parts from both neighbors' cars to build a new one



STEP 2: Crossover+Mutation — Combining Ideas From Neighbors

Friend 3 takes parts from both neighbors' cars to build a new one



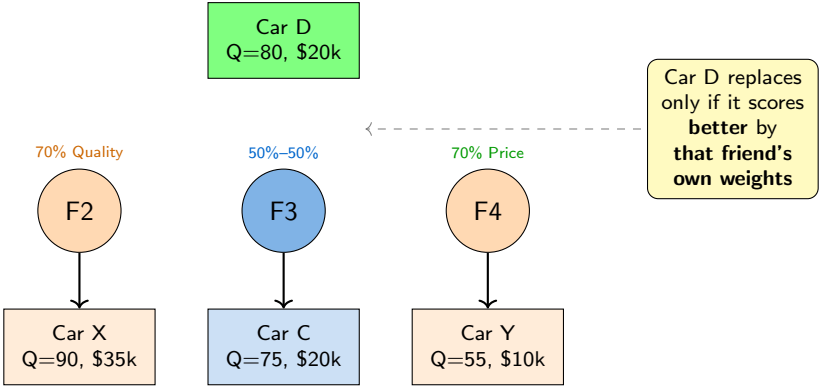
Step 3: Does Car D Actually Help the Neighbors?

Friend 3 shares Car D — each neighbor checks if it beats their current car

Car D
Q=80, $20k

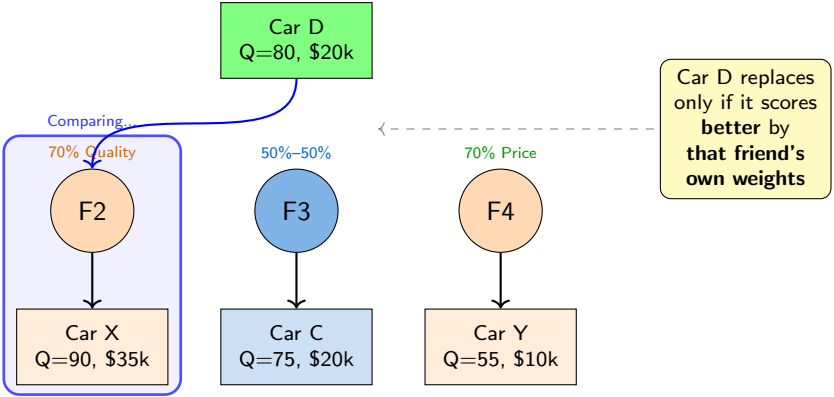
Step 3: Does Car D Actually Help the Neighbors?

Friend 3 shares Car D — each neighbor checks if it beats their current car



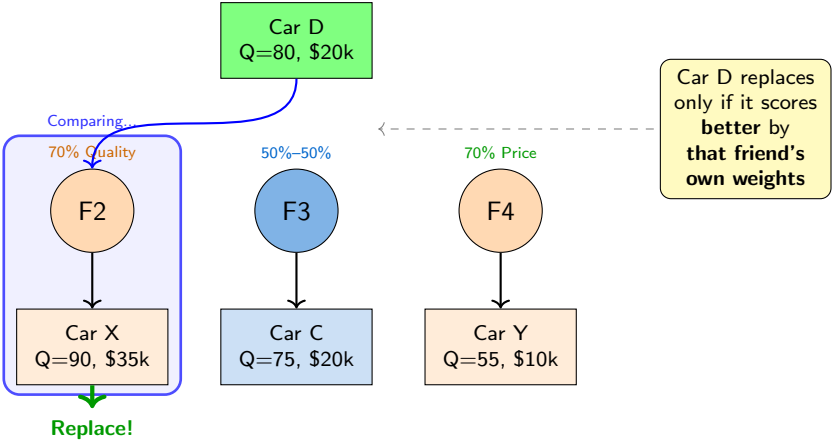
Step 3: Does Car D Actually Help the Neighbors?

Friend 3 shares Car D — each neighbor checks if it beats their current car



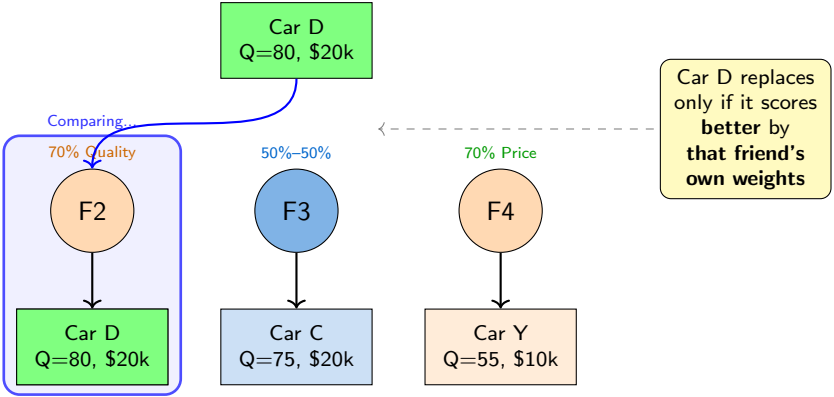
Step 3: Does Car D Actually Help the Neighbors?

Friend 3 shares Car D — each neighbor checks if it beats their current car



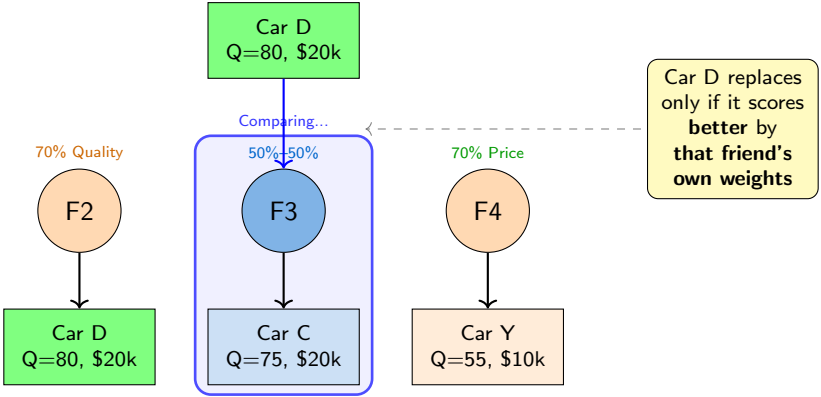
Step 3: Does Car D Actually Help the Neighbors?

Friend 3 shares Car D — each neighbor checks if it beats their current car



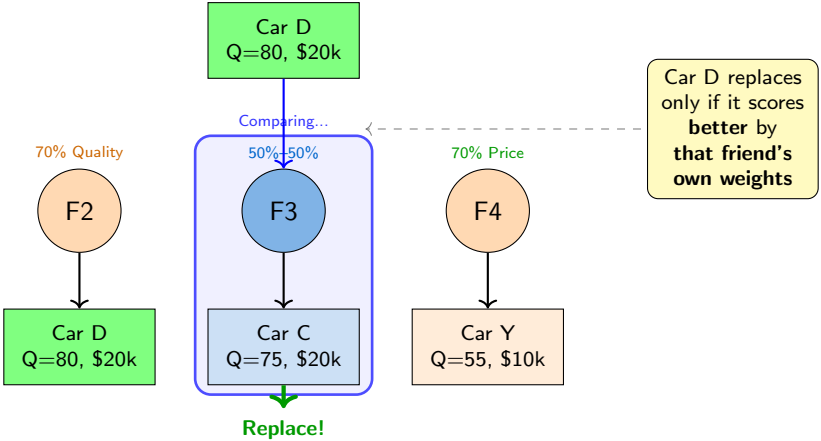
Step 3: Does Car D Actually Help the Neighbors?

Friend 3 shares Car D — each neighbor checks if it beats their current car



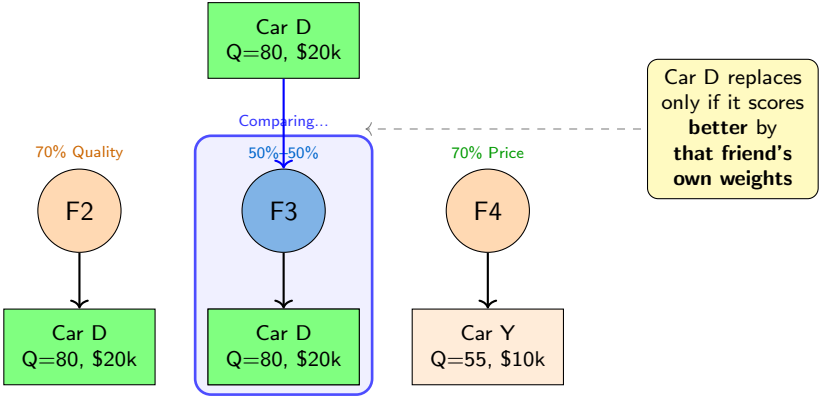
Step 3: Does Car D Actually Help the Neighbors?

Friend 3 shares Car D — each neighbor checks if it beats their current car



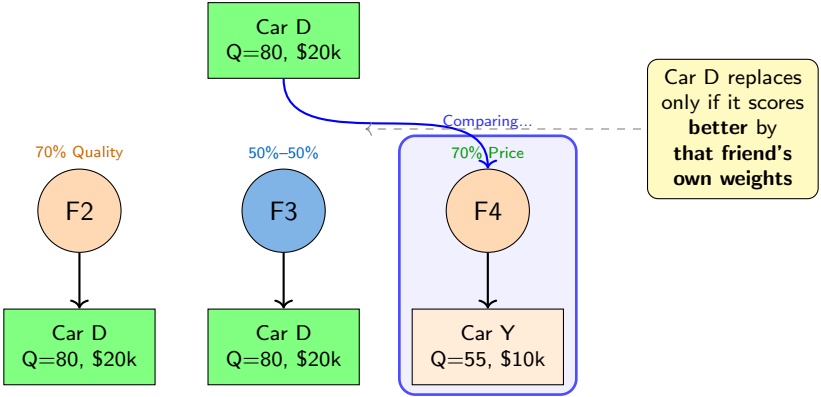
Step 3: Does Car D Actually Help the Neighbors?

Friend 3 shares Car D — each neighbor checks if it beats their current car



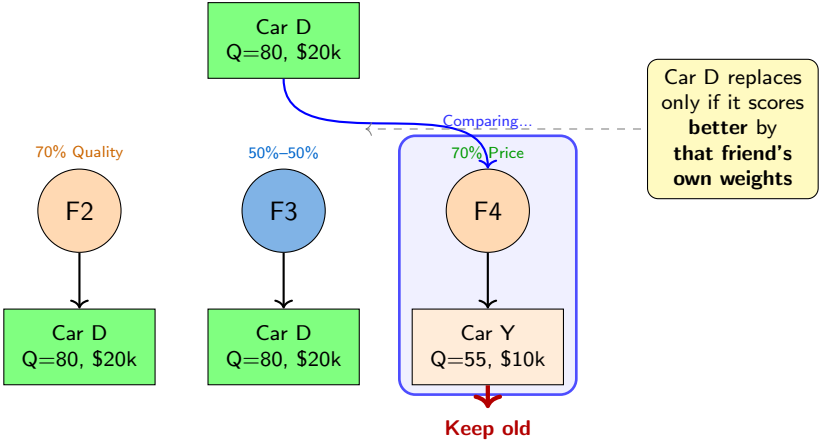
Step 3: Does Car D Actually Help the Neighbors?

Friend 3 shares Car D — each neighbor checks if it beats their current car



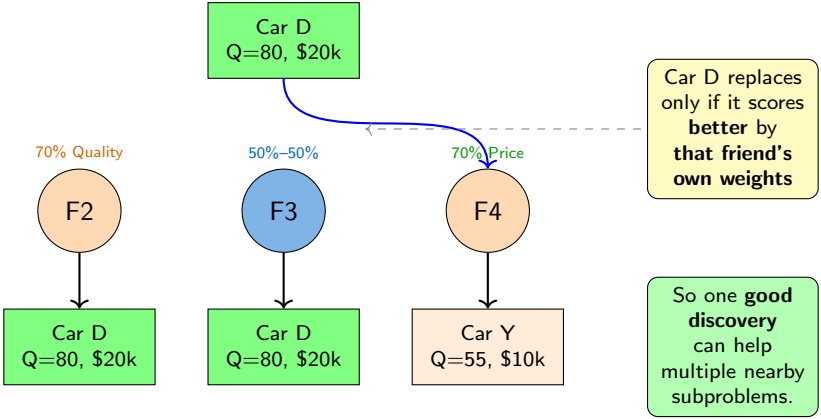
Step 3: Does Car D Actually Help the Neighbors?

Friend 3 shares Car D — each neighbor checks if it beats their current car



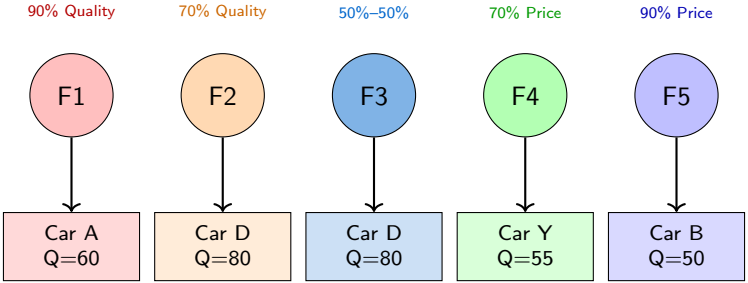
Step 3: Does Car D Actually Help the Neighbors?

Friend 3 shares Car D — each neighbor checks if it beats their current car



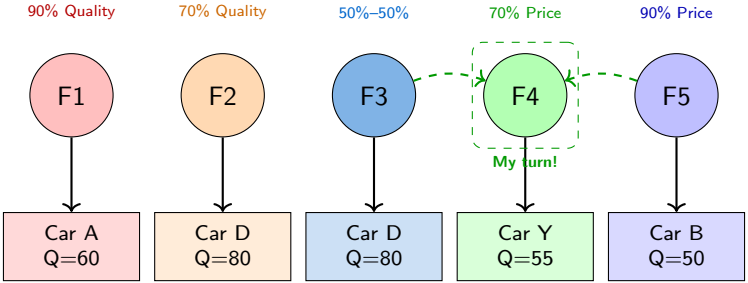
Step 4: Repeat Many Times — Everyone Keeps Improving

Each friend takes turns — look at neighbors, try new car, replace if better



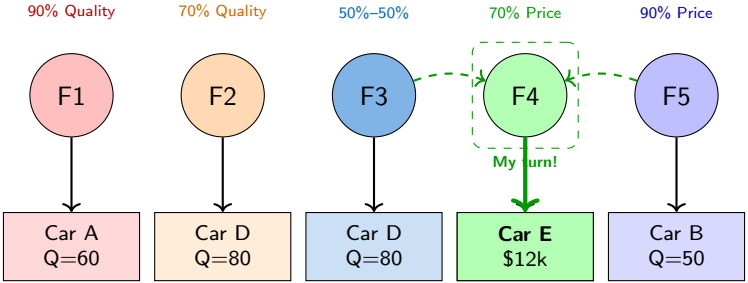
Step 4: Repeat Many Times — Everyone Keeps Improving

Each friend takes turns — look at neighbors, try new car, replace if better



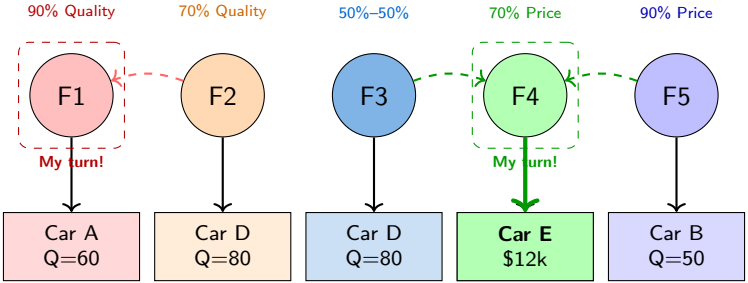
Step 4: Repeat Many Times — Everyone Keeps Improving

Each friend takes turns — look at neighbors, try new car, replace if better



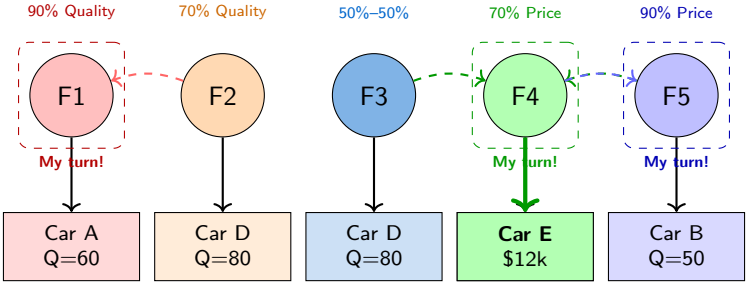
Step 4: Repeat Many Times — Everyone Keeps Improving

Each friend takes turns — look at neighbors, try new car, replace if better



Step 4: Repeat Many Times — Everyone Keeps Improving

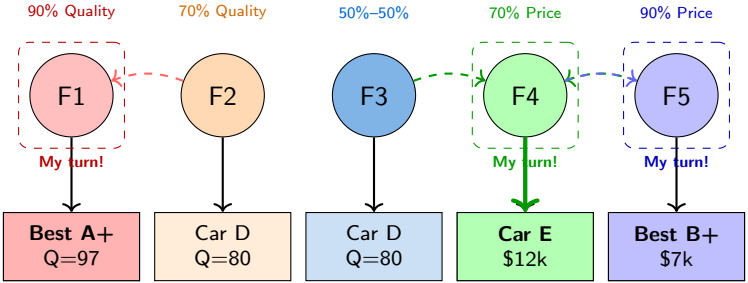
Each friend takes turns — look at neighbors, try new car, replace if better



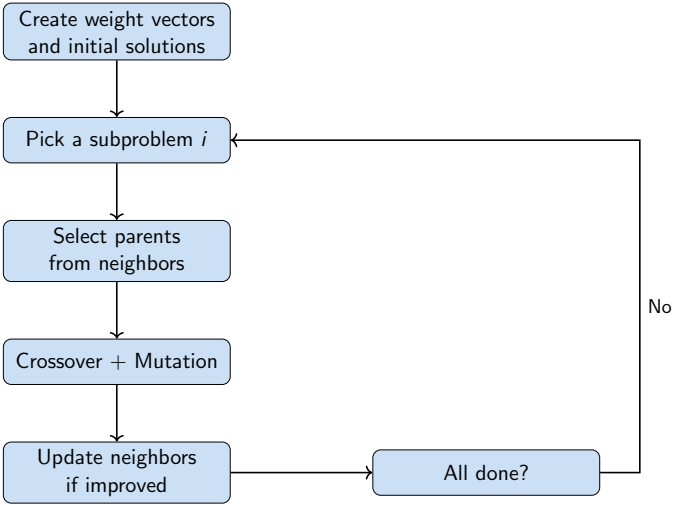
Step 4: Repeat Many Times — Everyone Keeps Improving

Each friend takes turns — look at neighbors, try new car, replace if better

After many rounds → each friend has the **best possible car** for *their own preference* = one point on the **Pareto front**



Why is MOEA/D Evolutionary? — The Main Loop



Yes → Pareto Front!

Why is MOEA/D Evolutionary? — The Main Loop

Each full loop = one **generation**. Many generations → population **evolves** toward the Pareto front.

Initial Population

Create weight vectors and initial solutions

↓

Pick a subproblem i

↓

Reproduction

Select parents from neighbors

↓

Crossover + Mutation

Crossover + Mutation

↓

Natural Selection

Update neighbors if improved

→

All done?

No =*Next Generation*

Yes → Pareto Front!

Conclusion

"Nothing is particularly hard if you divide it into small jobs."

– Henry Ford

Thank You!